

Apache Spark

CA4022: Alessandra Mileo alessandra.mleo@dcu.ie

Our Spark journey

- Part 1: Spark Introduction
- Part 2: RDDs
- Part 3: Using Spark
- Part 4: Machine Learning with Spark

Our Spark journey

- Part 1: Spark Introduction (a quick summary, see full presentation for more)
- Part 2: RDDs
- Part 3: Using Spark
- Part 4: Machine Learning with Spark

Part 1 – Spark Introduction

Classic Hadoop/MapReduce

- Hadoop black-box:
 - Read Process Write (from/to stable storage)
 - Difficult to access/manipulate data while processed
- File operations are slow!
- In-memory operations are faster

What is Apache Spark?

- **Fast, In-memory** Cluster computing for Big Data
- Combines
 - Batch, Stream, Interactive computations (via shell)
- Language Integration
 - Preferably **Scala** (compiled into JVM bytecode)
 - Also supports Java, Python, R, SQL (via API)
- Fully compatible with Hadoop/HDFS/Yarn
- Also works standalone (locally)

What is Apache Spark?

Fast and Expressive Cluster Computing
Engine Compatible with Apache Hadoop

Up to **10x** faster on disk,
100x in memory

Efficient

2-5x less code

Usable

- General execution graphs
- In-memory storage

- Rich APIs in Java, Scala, Python, R
- Interactive shell

Wordcount code for Hadoop

```
public class WordCount2 {

    public static class TokenizerMapper

        extends Mapper<Object, Text, Text, IntWritable>{

        static enum CountersEnum { INPUT_WORDS }

        private final static IntWritable one = new IntWritable(1);

        private Text word = new Text();

        private boolean caseSensitive;

        private Set<String> patternsToSkip = new HashSet<String>();

        private Configuration conf;

        private BufferedReader fis;

        @Override

        public void setup(Context context) throws IOException,

            InterruptedException {

            conf = context.getConfiguration();

            caseSensitive = conf.getBoolean("wordcount.case.sensitive", true);

            if (conf.getBoolean("wordcount.skip.patterns", true)) {

                URI[] patternsURIs = Job.getInstance(conf).getCacheFiles();

                for (URI patternsURI : patternsURIs) {

                    Path patternsPath = new Path(patternsURI.getPath());

                    String patternsFileName = patternsPath.getName().toString();

                    parseSkipFile(patternsFileName);

                }

            }

        }

    }

}
```

```
private void parseSkipFile(String fileName) {
    try {
        fis = new BufferedReader(new FileReader(fileName));
        String pattern = null;
        while ((pattern = fis.readLine()) != null) {
            patternsToSkip.add(pattern);
        }
    } catch (IOException ioe) {
        System.err.println("Caught exception while parsing the cached file "
            + StringUtils.stringifyException(ioe));
    }
}

@Override
public void map(Object key, Text value, Context context
    ) throws IOException, InterruptedException {
    String line = (caseSensitive) ?
        value.toString() : value.toString().toLowerCase();
    for (String pattern : patternsToSkip) {
        line = line.replaceAll(pattern, "");
    }
    StringTokenizer itr = new StringTokenizer(line);
    while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
        Counter counter = context.getCounter(CountersEnum.class.getName(),
            CountersEnum.INPUT_WORDS.toString());
        counter.increment(1);
    }
}

public static class IntSumReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context
        ) throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

```
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    GenericOptionsParser optionParser = new GenericOptionsParser(conf, args);
    String[] remainingArgs = optionParser.getRemainingArgs();
    if (!remainingArgs.length != 2 || remainingArgs.length != 4) {
        System.err.println("Usage: wordcount <in> <out> [-skip skipPatternFile]");
        System.exit(2);
    }
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount2.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    List<String> otherArgs = new ArrayList<String>();
    for (int i=0; i < remainingArgs.length; ++i) {
        if ("-skip".equals(remainingArgs[i])) {
            job.addCacheFile(new Path(remainingArgs[++i]).toUri());
            job.getConfiguration().setBoolean("wordcount.skip.patterns", true);
        } else {
            otherArgs.add(remainingArgs[i]);
        }
    }
    FileInputFormat.addInputPath(job, new Path(otherArgs.get(0)));
    FileOutputFormat.setOutputPath(job, new Path(otherArgs.get(1)));

    System.exit((job.waitForCompletion(true) ? 0 : 1);
}
```

Code from <http://baweaver.com/>

Wordcount with Spark and Scala

An example

```
val textFile = sc.textFile("hdfs://...")  
val counts = textFile.flatMap(line => line.split(" "))  
                      .map(word => (word, 1))  
                      .reduceByKey(_ + _)  
counts.saveAsTextFile("hdfs://...")
```

Spark Distributed Data Structures

- **RDD... (our focus)**: work well with schema-free data, not most efficient due to overhead for serialization
- DataFrame (since Spark 1.3): data organised in named columns (like tables); fast, stable, close to SQL, works best with Scala
- DataSets (since Spark 2.0): combines RDD and DataFrames, Only available in Scala and Java

Structure of Spark

Spark Core

Spark SQL

Spark
Streaming

Mlib

GraphX

Spark Streaming Vs. Storm

- **Storm** is a stream processing framework that supports micro-batching (with Trident), and it is a specialization of MR
- **Spark** is a batch processing framework that also does micro-batching (Spark Streaming) and it is a generalization of MR

End of Part 1 – Spark Introduction